

# What is Software?

Software is a program that enables a computer to perform a specific task. Software acts as an interface between the user and the computer hardware.

## Functions of Software

- Provides instructions for the computer to perform tasks.
- Enables communication between users and hardware.
- Helps manage and control computer resources.
- Allows applications to run properly.

## Major Classes of Software

### 1. System Software

System software manages and controls computer hardware and provides a platform for other software to run.

**Examples:** Windows, Linux, macOS.

### 2. Programming Software

Programming software provides tools for developers to create, test, debug, and maintain programs.

**Examples:** Compiler, Interpreter, Visual Studio.

### 3. Application Software

Application software is designed to help users perform specific tasks.

**Examples:** Microsoft Word, Microsoft Excel, Google Chrome, Adobe Photoshop.

## Easy Exam Definition

**"Software is a collection of programs, instructions, and data that enables a computer to perform specific tasks."**

## Memory Trick

**SPA = System Software + Programming Software + Application Software ✓**

## What is Software Testing?

Software testing is a process of checking and evaluating the functionality of a software application with an intent to find whether the developed software met the specified requirements or not and to identify the defects to ensure that the product is defect free in order to produce the quality product.

According to **ANSI/IEEE 1059**, software testing is the process of analyzing software to identify differences between expected and actual results (defects) and to evaluate its features.

## Quality Software:

Quality software refers to a software which is reasonably bug or defect free, is delivered in time and within the specified budget, meets the requirements and/or expectations, and is maintainable.

## Types of Software Quality

### 1. Functional Quality

It measures how well the software performs the functions required by users according to the specifications.

**Example:** A banking app correctly transfers money and shows account balance.

### 2. Structural Quality

It refers to the non-functional aspects of software such as maintainability, reliability, robustness, and efficiency.

**Example:** Software is easy to modify and continues to work properly even under heavy load.

## Software Quality Assurance (SQA)

Software Quality Assurance (SQA) is a set of activities to ensure the quality in software engineering processes that ultimately result in quality software products. The activities establish and evaluate the processes that produce products. It involves process-focused action.

## Software Quality Control (SQC)

**Software Quality Control (SQC) is a set of activities to ensure the quality in software products. These activities focus on determining the defects in the actual products produced. It involves product-focused action.**

## Comparison: Software Products vs Industrial Products (Easy Version)

| Characteristic | Software Products                         | Industrial Products                       |
|----------------|-------------------------------------------|-------------------------------------------|
| Complexity     | Has millions of operational possibilities | Has only thousands of operational options |

| Characteristic          | Software Products                                                 | Industrial Products                                                                    |
|-------------------------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <b>Visibility</b>       | Software is <b>invisible</b> , so defects cannot be seen directly | Industrial products are <b>visible</b> , so defects can be seen easily                 |
| <b>Defect Detection</b> | Defects are found mainly during <b>development/testing phase</b>  | Defects can be found in <b>all phases (design, production planning, manufacturing)</b> |

### One-Line Memory:

Software is **complex and invisible**, so **testing is harder**; industrial products are **visible and easier to inspect**. ✓

## How we will get quality software?

### 1. Clear Requirements

Just as we need to know how many floors or rooms a house should have before building it, we must clearly understand and document the client's requirements before development starts. If the requirements are wrong, the software will not be useful, no matter how fast it is.

### 2. Solid Architecture & Design

Just as a house needs a proper blueprint from an architect, software needs a strong architecture and database design before development begins. This helps the system remain stable even when the number of users grows.

### 3. Following SDLC & Standards

Developers should follow a standard **Software Development Life Cycle (SDLC)** process (such as Agile) and maintain clean coding standards. Poorly written code becomes difficult to maintain and update later.

### 4. Rigorous Testing

Just as a house is inspected after construction to ensure electricity and water work properly, software must be thoroughly tested through **Unit Testing, Integration Testing, and System Testing** to find and fix bugs.

### 5. User Involvement

Just as homeowners regularly check the progress of their house construction, users or clients should stay involved throughout development and provide feedback to ensure the software meets their needs.

## 6. Proper Maintenance

The job is not finished after delivering the software. Regular maintenance is necessary to fix issues, improve performance, and provide updates when needed.

## Short Exam Answer

Quality software can be achieved through clear requirements, solid design, following SDLC standards, rigorous testing, user involvement, and proper maintenance. ✓

### Memory Trick:

Requirements → Design → Coding Standards → Testing → User Feedback → Maintenance

### Quality Software Pabar Upay

- **Clear Requirements (সঠিক চাহিদা বোঝা):** Bari toiri korar aage jemon malik er theke shune nite hoy koto tola hobe ba koyta room thakbe; temni code lekhar aage client er ashol chahida (requirements) ekdom clear kore document korte hobe. Requirement vul hole software jotoi fast hok, kono kajer hobe na.
- **Solid Architecture & Design (মজবুত নকশা):** Bari bananor aage jemon architect er naksha lage, temni development shuru korar aage software er ekti solid architecture o database design dar korate hobe. Ete bhabishotte user barleo system crash korbe na.
- **Following SDLC & Standards (নিয়ম মেনে কোডিং):** Ekti standard Software Development Life Cycle (SDLC) process (jemon Agile) ebong clean coding standard mene code likhte hobe. Elo-melo code likhle pore maintain kora kothin hoye jay.
- **Rigorous Testing (কঠোর টেস্টিং):** Bari toiri sheshe jemon check kora hoy current o pani thikmoto asche kina, temni software toiri hobar por Unit Testing, Integration Testing, ebong System Testing kore shob bug (vul) solve korte hobe.
- **User Involvement (ইউজারদের সম্পৃক্ততা):** Barite jara thakbe tader jemon majhe majhe eshe dekhte hoy kaj thik hocche kina, temni bananor shomoy continuous user ba client der sathe communication rakhte hobe tader feedback neyar jonno.
- **Proper Maintenance (নিয়মিত রক্ষণাবেক্ষণ):** Software toiri kore delivery dilei kaj shesh hoy; chalu korar por kono issue ashle ta fix kora ebong notun update dewar jonno proper maintenance nishchit korte hobe.

### Software Fault, Error & Failure (Easy English)

In software, there are three important terms: **Fault, Error, and Failure.**

## 1. Software Fault

A **software fault** is a **mistake or defect in the software code or design**.

- ✓ It exists inside the software
- ✓ It is also called a bug

**Easy meaning:**

☞ Fault = a mistake in the software code or design

## 2. Software Error

A **software error** is an **incorrect internal state of the software caused by a fault**.

- ✓ It happens inside the system
- ✓ The program starts behaving incorrectly internally

**Easy meaning:**

☞ Error = wrong internal condition caused by a fault

## 3. Software Failure

A **software failure** is when the software shows **incorrect output or wrong behavior to the user**.

- ✓ It is visible to the user
- ✓ It does not meet requirements

**Easy meaning:**

☞ Failure = wrong result seen by user

## What is a Computer Bug?

A **computer bug** is a **mistake or problem in a computer program** that causes it to work incorrectly or produce wrong results.

**Bug is also called:**

- Defect
- Fault
- Error
- Problem
- Failure
- Anomaly
- Incident
- Inconsistency

- Product anomaly

## Defective Software

**Defective software** means software that contains **errors, bugs, or faults**.

Almost every software we develop has some defects, and it is difficult to make it 100% perfect.

☞ It is very hard to predict all future problems, so software may still contain defects even after testing.

## Sources of Problems in Software

- **Requirements Definition:** Erroneous, incomplete, or inconsistent requirements.
- **Design:** Fundamental design flaws in the software.
- **Implementation:** Mistakes in chip fabrication, wiring, programming faults, malicious code.
- **Support Systems:** Poor programming languages, faulty compilers and debuggers, misleading development tools.
- **Inadequate Testing of Software:** Incomplete testing, poor verification, mistakes in debugging.
- **Evolution:** Sloppy maintenance or redevelopment, new bugs while fixing old ones, increasing system complexity over time.

## মনে রাখার ট্রিক:

☞ R D I S T E

R = Requirements

D = Design

I = Implementation

S = Support Systems

T = Testing

E = Evolution ✓

### 1. Requirements Definition (চাহিদা নির্ধারণ)

ভুল, অসম্পূর্ণ বা অসামঞ্জস্যপূর্ণ (inconsistent) requirements-এর কারণে সমস্যা তৈরি হয়।

### 2. Design (নকশা)

Software-এর architecture বা design-এ মৌলিক ত্রুটি (design flaw) থাকলে সমস্যা দেখা দেয়।

### 3. Implementation (বাস্তবায়ন/কোডিং)

Programming mistakes, coding faults, wiring errors, chip fabrication mistakes অথবা malicious code-এর কারণে সমস্যা হতে পারে।

#### 4. Support Systems (সহায়ক সিস্টেম)

দুর্বল programming language, faulty compiler, debugger অথবা বিভ্রান্তিকর development tools-এর কারণে সমস্যা সৃষ্টি হতে পারে।

#### 5. Inadequate Testing of Software (অপর্যাপ্ত টেস্টিং)

অসম্পূর্ণ testing, দুর্বল verification এবং debugging-এর ভুলের কারণে software-এ bug থেকে যেতে পারে।

#### 6. Evolution (রক্ষণাবেক্ষণ ও উন্নয়ন)

Maintenance বা redevelopment-এর সময় অসাবধানতার কারণে নতুন bug তৈরি হতে পারে এবং software ধীরে ধীরে বেশি জটিল হয়ে যেতে পারে।

### Adverse Effects of Faulty Software

- **Communications:** Loss or corruption of communication media, non-delivery of data.
- **Space Applications:** Loss of lives, launch delays.
- **Defense and Warfare:** Misidentification of friend or foe.
- **Transportation:** Deaths, delays, sudden acceleration, inability to brake.
- **Safety-critical Applications:** Death, injuries.
- **Electric Power:** Death, injuries, power outages, long-term health hazards (radiation).
- **Money Management:** Fraud, privacy violation, shutdown of stock exchanges and banks, negative interest rates.
- **Control of Elections:** Wrong results (intentional or unintentional).
- **Control of Jails:** Technology-aided escape attempts, accidental release of inmates, failure of software-controlled locks.
- **Law Enforcement:** False arrests and wrongful imprisonments.

Short Technique:

-----  
**CSDTSEMJL**

#### 1. Communications (যোগাযোগ ব্যবস্থা)

- তথ্য হারিয়ে যাওয়া (Data Loss)
- তথ্য বিকৃত হয়ে যাওয়া (Data Corruption)
- তথ্য সঠিকভাবে পৌঁছাতে না পারা

## 2. Space Applications (মহাকাশ কার্যক্রম)

- প্রাণহানি
- রকেট/মিশন উৎক্ষেপণে বিলম্ব

## 3. Defense and Warfare (প্রতিরক্ষা ও যুদ্ধ)

- বন্ধু ও শত্রুকে ভুলভাবে শনাক্ত করা

## 4. Transportation (পরিবহন ব্যবস্থা)

- দুর্ঘটনা ও প্রাণহানি
- যাত্রায় বিলম্ব
- গাড়ির হঠাৎ গতি বেড়ে যাওয়া
- ব্রেক কাজ না করা

## 5. Safety-Critical Applications (জীবন-নির্ভর গুরুত্বপূর্ণ সিস্টেম)

- মৃত্যু
- আহত হওয়া

## 6. Electric Power (বিদ্যুৎ ব্যবস্থা)

- মৃত্যু ও আহত হওয়া
- বিদ্যুৎ বিভ্রাট (Power Outage)
- দীর্ঘমেয়াদি স্বাস্থ্যঝুঁকি (যেমন: Radiation)

## 7. Money Management (আর্থিক ব্যবস্থাপনা)

- জালিয়াতি (Fraud)
- ব্যক্তিগত তথ্যের গোপনীয়তা লঙ্ঘন
- ব্যাংক ও শেয়ারবাজার বন্ধ হয়ে যাওয়া
- নেতিবাচক সুদের হার (Negative Interest Rates)

## 8. Control of Elections (নির্বাচন ব্যবস্থা)

- ভুল নির্বাচনী ফলাফল (ইচ্ছাকৃত বা অনিচ্ছাকৃত)

## 9. Control of Jails (কারাগার ব্যবস্থা)

- প্রযুক্তির সাহায্যে পালানোর চেষ্টা বা সফল হওয়া



- ভুলবশত বন্দিদের মুক্তি দেওয়া
- সফটওয়্যার নিয়ন্ত্রিত তালার ব্যর্থতা

## 10. Law Enforcement (আইন প্রয়োগকারী সংস্থা)

- নির্দোষ ব্যক্তিকে ভুলভাবে গ্রেফতার করা
- অন্যায়ভাবে কারাবন্দী করা

Just Mone rakhar jonno:

-----

- **Communications** → Data loss
- **Space Applications** → Life loss, launch delay
- **Defense & Warfare** → Wrong target identification
- **Transportation** → Accidents, brake failure
- **Safety-critical Applications** → Death, injuries
- **Electric Power** → Power outage, radiation risk
- **Money Management** → Fraud, privacy violation
- **Control of Elections** → Wrong election results
- **Control of Jails** → Prisoner escape/release
- **Law Enforcement** → False arrest

## Verification and Validation

### Verification

**Verification** is the process of checking whether the software is being developed correctly according to the requirements of the previous phase.

☞ **Easy Meaning:**

"Are we building the product right?"

### Validation

**Validation** is the process of checking whether the final software meets the user's needs and intended use.

☞ **Easy Meaning:**

"Are we building the right product?"

## Short Definitions for Exam

**Verification:**

Checking whether each development phase meets the requirements of the previous phase.

**Validation:**

Checking whether the final software satisfies the user's requirements and intended use.

**Easy Difference**

| Verification                         | Validation                           |
|--------------------------------------|--------------------------------------|
| Checks the development process       | Checks the final product             |
| "Are we building the product right?" | "Are we building the right product?" |
| Done during development              | Done after development               |

## Testing Goals Based on Test Process Maturity Level (Full Marks Easy Version)

এখানে দেখানো হয়েছে Testing সম্পর্কে মানুষের চিন্তাধারা (Thinking) কীভাবে ধীরে ধীরে উন্নত হয়েছে।

### Level 0 Thinking: Testing = Debugging

**Main Idea:**

Testing এবং Debugging-কে একই জিনিস মনে করা হয়।

**Easy Explanation:**

- Bug খোঁজা এবং Bug ঠিক করাকেই Testing ধরা হয়।
- Software-এর ভুল আচরণ (Failure) এবং Program-এর ভুল (Fault) এর মধ্যে পার্থক্য করা হয় না।
- এতে Reliable ও Safe Software তৈরি করা কঠিন।
- সাধারণত Undergraduate CS Students-দের শুরুতে এভাবেই শেখানো হয়।

**Exam Point:**

Testing is considered the same as debugging and does not help build reliable software.

### Level 1 Thinking: Show Correctness

## Main Idea:

Testing-এর উদ্দেশ্য হলো Software সঠিক (Correct) তা প্রমাণ করা।

## Easy Explanation:

- 100% Correctness প্রমাণ করা প্রায় অসম্ভব।
- যদি কোনো Failure না পাওয়া যায়, তাহলে প্রশ্ন আসে:
  - Software কি সত্যিই ভালো?
  - নাকি Test-গুলোই দুর্বল ছিল?
- Test Engineers-এর:
  - Clear Goal নেই
  - Stopping Rule নেই
  - Formal Testing Technique নেই
- Test Managers-এর ক্ষমতাও সীমিত।
- Hardware Engineers অনেক সময় এমনটাই আশা করেন।

## Exam Point:

**The goal is to prove software correctness, but complete correctness can never be guaranteed.**

## Level 2 Thinking: Show Failures

### Main Idea:

Testing-এর উদ্দেশ্য হলো Failure বা Bug খুঁজে বের করা।

### Easy Explanation:

- Tester-এর কাজ হলো ভুল খুঁজে বের করা।
- Testing একটি Negative Activity হয়ে যায়।
- Tester এবং Developer-এর মধ্যে বিরোধ (Adversarial Relationship) তৈরি হয়।
- যদি কোনো Failure না পাওয়া যায়, তখনও নিশ্চিত হওয়া যায় না যে Software সম্পূর্ণ সঠিক।
- অধিকাংশ Software Company এই Level-এ কাজ করে।

## Exam Point:

**The purpose of testing is to find failures, which may create conflict between testers and developers.**

## Level 3 Thinking: Reduce Risk

### Main Idea:

Testing-এর কাজ Software Perfect প্রমাণ করা নয়, বরং Risk কমানো।

### Easy Explanation:

- Testing শুধু Failure-এর উপস্থিতি দেখাতে পারে।
- Software ব্যবহার করলে সবসময় কিছু Risk থাকে।
- Risk:
  - ছোট হতে পারে
  - আবার ভয়াবহ (Catastrophic) হতে পারে
- Tester এবং Developer একসাথে কাজ করে Risk কমায়।
- কিছু উন্নত (Enlightened) Company এই Level অনুসরণ করে।

### Exam Point:

Testing helps reduce the risk of using software through cooperation between testers and developers.

## Level 4 Thinking: Improve Quality

### Main Idea:

Testing হলো একটি Mental Discipline যা Software Quality বৃদ্ধি করে।

### Easy Explanation:

- Testing Quality বাড়ানোর একটি উপায়।
- Test Engineers Project-এর Technical Leader হতে পারেন।
- তাদের মূল দায়িত্ব:
  - Software Quality Measure করা
  - Software Quality Improve করা
- তারা Developers-দের সাহায্য করে ভালো Software তৈরি করতে।
- Traditional Engineering-এ এভাবেই কাজ করা হয়।

### Exam Point:

Testing is a quality improvement discipline that helps develop high-quality software.

---

# Easy Summary Table

| Level   | Goal                     |
|---------|--------------------------|
| Level 0 | Testing = Debugging      |
| Level 1 | Show Correctness         |
| Level 2 | Find Failures            |
| Level 3 | Reduce Risk              |
| Level 4 | Improve Software Quality |

## Super Easy Memory Trick

- ☞ 0 = Debug
- ☞ 1 = Correct
- ☞ 2 = Failures
- ☞ 3 = Risk
- ☞ 4 = Quality

### Shortcut:

Debug → Correct → Failures → Risk → Quality ✓

এভাবে লিখলে Level 0 থেকে Level 4-এর কোনো গুরুত্বপূর্ণ পয়েন্ট মিস হবে না এবং পরীক্ষায় full marks পাওয়ার মতো উত্তর হবে।

## Testing Goals Based on Test Process Maturity Level

This model shows how the understanding and purpose of software testing have evolved over time.

### Level 0 Thinking: Testing = Debugging

#### Main Idea:

Testing and debugging are considered the same.

#### Easy Explanation:

- Finding and fixing bugs is considered testing.
- No distinction is made between software failures and programming mistakes.
- It does not help develop reliable or safe software.

- This is commonly how undergraduate Computer Science students are initially taught.

**Exam Point:**

Testing is considered the same as debugging and does not help build reliable software.

## Level 1 Thinking: Show Correctness

**Main Idea:**

The purpose of testing is to prove that the software is correct.

**Easy Explanation:**

- Complete correctness is impossible to prove.
- If no failures are found, we cannot be sure whether:
  - The software is good, or
  - The tests were poor.
- Test engineers have:
  - No strict goal
  - No real stopping rule
  - No formal testing technique
- Test managers have little control.
- This is often what hardware engineers expect.

**Exam Point:**

The goal is to prove software correctness, but complete correctness can never be guaranteed.

## Level 2 Thinking: Show Failures

**Main Idea:**

The purpose of testing is to find failures and defects.

**Easy Explanation:**

- Testers focus on finding bugs and failures.
- Testing becomes a negative activity.
- Testers and developers may develop an adversarial relationship.
- If no failures are found, we still cannot guarantee that the software is perfect.
- Most software companies work at this level.

**Exam Point:**

**The purpose of testing is to find failures, which may create conflict between testers and developers.**

## **Level 3 Thinking: Reduce Risk**

**Main Idea:**

The purpose of testing is to reduce the risk of using the software.

**Easy Explanation:**

- Testing can only show the presence of failures, not their absence.
- Every software system involves some risk.
- Risks may be:
  - Small and unimportant, or
  - Serious and catastrophic.
- Testers and developers work together to reduce risk.
- A few advanced software companies follow this approach.

**Exam Point:**

**Testing helps reduce the risk of using software through cooperation between testers and developers.**

## **Level 4 Thinking: Improve Quality**

**Main Idea:**

Testing is a mental discipline that improves software quality.

**Easy Explanation:**

- Testing is one way to improve software quality.
- Test engineers can become technical leaders of a project.
- Their main responsibility is:
  - Measuring software quality
  - Improving software quality
- Their expertise helps developers create better software.
- This is how traditional engineering disciplines work.

**Exam Point:**

Testing is a quality improvement discipline that helps develop high-quality software.

## Easy Summary Table

| Level           | Goal                     |
|-----------------|--------------------------|
| Level 0 Testing | = Debugging              |
| Level 1         | Show Correctness         |
| Level 2         | Find Failures            |
| Level 3         | Reduce Risk              |
| Level 4         | Improve Software Quality |

### পর্ব ১: Software Testing Model (টেস্টিং এর ডায়াগ্রাম)

এই ডায়াগ্রামে একটি সফটওয়্যার টেস্ট করার প্রক্রিয়া বা ফ্লো দেখানো হয়েছে। ধরুন আপনি একটি ক্যালকুলেটর অ্যাপ বানিয়েছেন এবং সেটি টেস্ট করছেন:

- **Test Data (টেস্ট ডেটা):** এটি হলো আপনার দেওয়া ইনপুট। যেমন: আপনি টেস্ট করার জন্য ক্যালকুলেটরে  $2 + 2$  দিলেন।
- **Software under Test (সফটওয়্যার):** এটি হলো আপনার আসল কোড বা সফটওয়্যার (ক্যালকুলেটর অ্যাপটি) যাকে আপনি টেস্ট করছেন।
- **Output (আউটপুট):** আপনার সফটওয়্যার প্রসেস করার পর যে রেজাল্ট বা ফলাফলটি দেয়। যেমন: অ্যাপটি হিসাব করে ৪ (বা কোডে কোনো ত্রুটি থাকলে ৫) আউটপুট দিলো।
- **Expected Output (প্রত্যাশিত আউটপুট):** টেস্ট করার আগেই আপনি জানেন যে সঠিক রেজাল্ট কী আসা উচিত। এখানে প্রত্যাশিত আউটপুট হলো ৪।
- **Oracle (ওরাকল):** এটি হলো সেই ব্যবস্থা বা মেকানিজম যা আসল **Output** এবং **Expected Output**-কে তুলনা (compare) করে। ওরাকল যদি দেখে দুটোই হুবহু মিলে গেছে, তাহলে টেস্ট "Pass" (পাস), আর না মিললে "Fail" (ফেল)।

## Part 1: Software Testing Model

This diagram shows the **basic flow of software testing**. Let's understand it with a simple example of a **calculator app**.

### 1. Test Data

Test data means the **input we give for testing**.

☞ Example:

We enter  $2 + 2$  in the calculator.

### 2. Software under Test (SUT)



This is the **actual software or program** that we are testing.

☞ Example:

The calculator application (your code).

### 3. Output (Actual Result)

This is the **result given by the software after processing the input**.

☞ Example:

The calculator shows **4** (or sometimes wrong output like 5 if there is a bug).

### 4. Expected Output

This is the **correct result that we expect before testing**.

☞ Example:

For  $2 + 2$ , the expected output is **4**.

### 5. Oracle

Oracle is the **system or mechanism that compares actual output with expected output**.

- If both match → **Test Pass**
- If they do not match → **Test Fail**

## পর্ব ২: Anatomy of a Test (একটি টেস্টের ৪টি অংশ)

যেকোনো স্ট্যান্ডার্ড সফটওয়্যার টেস্টের ৪টি ধাপ বা অংশ থাকে। ল্যাবে সায়েন্স এক্সপেরিমেন্ট করার সাথে তুলনা করে এটি খুব সহজে মনে রাখা যায়:

- **১. Setup (প্রস্তুতি):** টেস্ট শুরু করার আগে প্রয়োজনীয় পরিবেশ বা এনভায়রনমেন্ট তৈরি করা। যেমন: দরকারি ডেটা লোড করা, ডাটাবেস কানেকশন ওপেন করা বা ভেরিয়েবল সেট করা।
  - (তুলনা: ল্যাবে পরীক্ষা শুরুর আগে টেবিল পরিষ্কার করে সব যন্ত্রপাতি গুছিয়ে নেওয়া)।
- **২. Invocation (চালানো বা কল করা):** আসল ফাংশন, মেথড বা কোডটি রান বা এক্সিকিউট (execute) করা।
  - (তুলনা: আসল এক্সপেরিমেন্টটি শুরু করা এবং কেমিক্যালগুলো মেশানো)।
- **৩. Assessment (মূল্যায়ন):** সফটওয়্যার থেকে পাওয়া রেজাল্টের (Output) সাথে আপনার আগে থেকে জানা সঠিক রেজাল্ট (Expected output) মিলিয়ে দেখা। এটি মূলত 'Oracle'-এর কাজ।

- (তুলনা: এক্সপেরিমেন্ট শেষে পাওয়া ফলাফলের সাথে বইয়ের সঠিক ফলাফল মিলিয়ে দেখা)।
- **8. Teardown (পরিষ্কার করা):** টেস্ট শেষ হওয়ার পর সবকিছু একদম আগের অবস্থায় ফিরিয়ে আনা, যাতে এর প্রভাব অন্য কোনো টেস্টের ওপর না পড়ে। যেমন: ওপেন করা ডাটাবেস কানেকশন ক্লোজ করে দেওয়া বা টেস্টের জন্য তৈরি করা ডামি ফাইলগুলো ডিলিট করে ফেলা।
  - (তুলনা: এক্সপেরিমেন্ট শেষে ল্যাবের টেবিল পরিষ্কার করে সব আগের মতো গুছিয়ে রাখা)।

## Part 2: Anatomy of a Test (Easy English)

Every standard software test has **4 main steps**. We can understand it easily by comparing it with a **science lab experiment**.

### 1. Setup (Preparation)

Before starting the test, we prepare everything needed for testing.

- Load test data
- Open database connections
- Set variables and environment

#### ☞ Easy meaning:

Prepare the system for testing.

#### □ Example comparison:

Like arranging all equipment before starting a lab experiment.

### 2. Invocation (Execution / Running)

This is the step where we actually **run the code or function**.

#### ☞ Easy meaning:

Execute the software or function that we want to test.

#### □ Example comparison:

Like starting the experiment and mixing chemicals.

### 3. Assessment (Checking Result)

In this step, we **compare the actual output with the expected output**.

☞ This is usually done by an **Oracle**.

☞ **Easy meaning:**

Check whether the result is correct or not.

□ **Example comparison:**

Like comparing experiment results with textbook answers.

## 4. Teardown (Cleanup)

After testing is finished, we **clean everything and restore the system**.

- Close database connections
- Delete test files
- Reset environment

☞ **Easy meaning:**

Bring the system back to its original state.

□ **Example comparison:**

Like cleaning the lab after the experiment.

## Easy Summary

☞ **Setup → Invocation → Assessment → Teardown**

## Fault & Failure Model (RIPR)

### RIPR Model (Very Easy English)

To find a **failure** in a software program, **4 conditions** must happen:

#### 1. Reachability

The test must **reach the faulty code**.

**Example:**

```
if (age > 18)
{
    // bug is here
}
```

If `age = 15`, this code does not run.

→ Fault is **not reached**.

## 2. Infection

The fault must make the **program state incorrect**.

**Example:**

```
total = price - quantity; // should be +
```

If price = 100 and quantity = 20,  
result becomes 80 instead of 120.

→ The program state is wrong.

→ Infection occurs.

## 3. Propagation

The incorrect state must **affect the final output**.

**Example:**

```
total = price - quantity; // wrong  
total = 120;               // corrected later
```

The error happened, but it did not reach the output.

→ No propagation.

## 4. Reveability

The tester must **notice the wrong output**.

**Example:**

Expected Output = 120

Actual Output = 80

If the tester does not check the result, the failure is not detected.

→ Error exists, but it is not revealed.

**R → I → P → R**

☞ **Reach → Infect → Propagate → Reveal**

ধরো, একটি প্রোগ্রামে bug (fault) আছে। Bug থাকলেই Failure দেখা যাবে না। Failure দেখার জন্য ৪টি শর্ত পূরণ হতে হবে।

## 1. Reachability (পৌঁছানো)

**Fault-এর কাছে পৌঁছাতে হবে।**

যদি টেস্ট করার সময় bug থাকা কোডই execute না হয়, তাহলে bug ধরা পড়বে না।

**উদাহরণ:**

```
if (age > 18)
{
    // এখানে bug আছে
}
```

তুমি যদি age = 15 দাও, তাহলে এই কোডে ঢুকবেই না।

→ Fault reached হয়নি।

## 2. Infection (সংক্রমণ)

**Fault execute হওয়ার পর প্রোগ্রামের state ভুল হতে হবে।**

Bug চলল, কিন্তু ফলাফলে কোনো ভুল হলো না—তাহলে Infection হয়নি।

**উদাহরণ:**

```
int total = price - quantity; // আসলে + হওয়ার কথা

price=100, quantity=20
```

Output হবে ৪০।

→ Program state ভুল হয়েছে।

→ Infection হয়েছে।

## 3. Propagation (ছড়িয়ে পড়া)

**ভুল state শেষ output পর্যন্ত পৌঁছাতে হবে।**

ভুল হয়েছে, কিন্তু পরে অন্য কোড সেটি ঠিক করে ফেলল।

## উদাহরণ:

```
total = price - quantity; // ভুল  
total = 100;              // পরে ঠিক করে দিল
```

এখানে ভুল হয়েছিল, কিন্তু output-এ আর দেখা যাচ্ছে না।

- Infection হয়েছে
- Propagation হয়নি

## 4. Revealability (প্রকাশ পাওয়া)

Tester-কে ভুল output দেখতে হবে।

Output ভুল এসেছে, কিন্তু tester সেটা check করল না।

## উদাহরণ:

Expected Output = 120

Actual Output = 80

কিন্তু tester শুধু "Program Runs Successfully" দেখল।

- Failure হয়েছে
- কিন্তু Reveal হয়নি

# V-Model (Easy Exam Version)

## Easy Idea of V-Model

In the **V-Model**, every **development phase** has a corresponding **testing phase**.

**Left Side = Development**

**Right Side = Testing**

So, whatever is designed on the left side is verified by testing on the right side.

## Easy Formula

| Development Phase | Testing Phase |
|-------------------|---------------|
|-------------------|---------------|

|                       |                    |
|-----------------------|--------------------|
| Requirements Analysis | Acceptance Testing |
|-----------------------|--------------------|

| Development Phase       | Testing Phase       |
|-------------------------|---------------------|
| Architectural Design    | System Testing      |
| Subsystem Design        | Integration Testing |
| Detailed Design         | Module Testing      |
| Implementation (Coding) | Unit Testing        |

# 1. Requirements Analysis ↔ Acceptance Testing

## English

During **Requirements Analysis**, customer needs and requirements are gathered.

**Acceptance Testing** checks whether the completed software satisfies the customer's requirements and expectations.

It answers the question:

**"Does the software do what the user wants?"**

Acceptance testing is usually performed by users or domain experts.

## Bangla

**Requirements Analysis** পর্যায়ে গ্রাহকের চাহিদা ও প্রয়োজনীয়তা সংগ্রহ করা হয়।

**Acceptance Testing** যাচাই করে সফটওয়্যারটি ব্যবহারকারীর চাহিদা পূরণ করছে কিনা।

এটি মূলত প্রশ্ন করে:

**"সফটওয়্যার কি ব্যবহারকারী যা চেয়েছে তা করছে?"**

এই টেস্ট সাধারণত ব্যবহারকারী বা ডোমেইন বিশেষজ্ঞরা করে।

# 2. Architectural Design ↔ System Testing

## English

During **Architectural Design**, the overall system structure, components, and connections are designed.

**System Testing** verifies whether the complete integrated system works according to the specifications.

It answers:

**"Does the whole system work correctly?"**

System testing usually focuses on design and specification issues.

## **Bangla**

**Architectural Design** পর্যায়ে পুরো সিস্টেমের কাঠামো, কম্পোনেন্ট ও তাদের সংযোগ নির্ধারণ করা হয়।

**System Testing** যাচাই করে সম্পূর্ণ সিস্টেম স্পেসিফিকেশন অনুযায়ী কাজ করছে কিনা।

এটি মূলত প্রশ্ন করে:

**"পুরো সিস্টেম কি সঠিকভাবে কাজ করছে?"**

এখানে সাধারণত ডিজাইন ও স্পেসিফিকেশন সংক্রান্ত ত্রুটি খোঁজা হয়।

# **3. Subsystem Design ↔ Integration Testing**

## **English**

During **Subsystem Design**, the structure and behavior of subsystems are defined.

**Integration Testing** checks whether different modules communicate correctly and whether their interfaces are compatible.

It answers:

**"Do the modules work together correctly?"**

Integration testing assumes that individual modules already work properly.

## **Bangla**

**Subsystem Design** পর্যায়ে বিভিন্ন সাবসিস্টেমের গঠন ও আচরণ নির্ধারণ করা হয়।



**Integration Testing** যাচাই করে মডিউলগুলো একে অপরের সাথে সঠিকভাবে যোগাযোগ করছে কিনা।

এটি মূলত প্রশ্ন করে:

"মডিউলগুলো কি একসাথে সঠিকভাবে কাজ করছে?"

এখানে ধরে নেওয়া হয় যে প্রতিটি মডিউল আলাদাভাবে ঠিকমতো কাজ করছে।

## 4. Detailed Design ↔ Module Testing

### English

During **Detailed Design**, the internal structure and behavior of individual modules are designed.

**Module Testing** checks whether each module works correctly in isolation.

It also verifies interactions among units and data structures inside the module.

Usually performed by developers.

### Bangla

**Detailed Design** পর্যায়ে প্রতিটি মডিউলের অভ্যন্তরীণ কাঠামো ও আচরণ নির্ধারণ করা হয়।

**Module Testing** একটি মডিউলকে আলাদাভাবে পরীক্ষা করে।

এটি মডিউলের ইউনিট ও ডেটা স্ট্রাকচারের পারস্পরিক কাজও যাচাই করে।

সাধারণত ডেভেলপাররা এই টেস্ট করে।

## 5. Implementation ↔ Unit Testing

### English

**Implementation** is the coding phase where actual program code is written.

**Unit Testing** is the lowest level of testing that checks individual functions, methods, or procedures.

It answers:

**"Does this single unit work correctly?"**

Usually performed by programmers.

## **Bangla**

**Implementation** হলো কোড লেখার পর্যায়ে।

**Unit Testing** সবচেয়ে নিচের স্তরের টেস্টিং, যেখানে একটি ফাংশন, মেথড বা প্রোসিডিউরকে পরীক্ষা করা হয়।

এটি মূলত প্রশ্ন করে:

**"এই একক ইউনিট কি সঠিকভাবে কাজ করছে?"**

সাধারণত প্রোগ্রামাররাই এটি করে।